

Prova I - Teoria

Entrega 29 set em 10:40**Pontos** 10**Perguntas** 14**Disponível** 29 set em 8:40 - 29 set em 10:40 aproximadamente 2 horas**Limite de tempo** 110 Minutos

Instruções

Nossa primeira prova de AEDs II tem duas partes: teórica e prática. Cada uma vale 10 pontos. A prova teórica será realizada no Canvas e a prática foi no Verde.

A prova teórica terá 14 questões. A primeira é para que você informe seu tempo de estudo na disciplina de AEDs II e vale 0,5 pontos. Em seguida, temos 11 questões fechadas e 2 abertas. Cada fechada vale 0.5 pontos e cada aberta, 2 pontos. Após terminar uma questão, o aluno **não** terá a opção de retornar à mesma. No caso das questões abertas, o aluno poderá enviar um arquivo contendo sua resposta. Essa resposta pode ser uma imagem (foto de boa resolução) ou um código fonte. No horário da aula, você pode acessar a Prova I através do menu "Testes / Prova I".

Este teste foi travado 29 set em 10:40.

Histórico de tentativas

	Tentativa	Tempo	Pontuação
MAIS RECENTE	Tentativa 1	106 minutos	7,5 de 10

❗ As respostas corretas não estão mais disponíveis.

Pontuação deste teste: **7,5** de 10

Enviado 29 set em 10:26

Esta tentativa levou 106 minutos.

Parcial

Pergunta 1

0,5 / 0,5 pts

Quantas horas você está estudando por semana para a disciplina de AEDs II? Todas as respostas serão consideradas corretas

- ☐ Menos de 5 horas
- ☐ Entre 10 e 15 horas
- ☐ Entre 5 e 10 horas.
- ☐ Mais de 20 horas
- ☒ Entre 15 e 20 horas

Pergunta 2

0,5 / 0,5 pts

Um desafio no projeto de algoritmos é a obtenção de um custo computacional reduzido. Para isso, uma habilidade do projetista é contar o número de operações realizadas pelo algoritmo. O trecho de código abaixo realiza algumas operações.

```
for (int i = n; i > 0; i >>= 1){  
    a *= 2;
```

}

Considerando o código acima, assinale a opção que apresenta a função de complexidade $f(n)$ para o melhor e pior caso considerando a operação de multiplicação.

☐☐☐☒☐

Pergunta 3

0,5 / 0,5 pts

Um desafio no projeto de algoritmos é a obtenção de um custo computacional reduzido. Para isso, o projetista de algoritmos deve ser capaz de contar o número de operações realizadas em seus algoritmos. O trecho de código abaixo realiza algumas operações.

```
for (int i = 2; i < n; i++){  
    for (int j = 2; j <= n; j++){  
        l = a * 2 + b * 5;  
    }  
}
```

Considerando o código acima, assinale a opção que apresenta a função de complexidade $f(n)$ para o melhor e pior caso considerando a operação número de multiplicações.

☐☐☒☐☐

Pergunta 4

0,5 / 0,5 pts

A contagem do número de operações realizadas por um algoritmo é uma tarefa fundamental para identificar seu custo computacional. O trecho de código abaixo realiza algumas operações.

```
Random gerador = new Random();
gerador.setSeed(4);
int x = Math.abs(gerador.nextInt());

for (int i = 0; i < n-4; i++){
    if( x % 9 < 4) {
        a *= 2; b *= 3; l *= 2;
    } else if (x % 9 == 5) {
```

```
    a *= 2; l *= 3;
  } else {
    a *= 2;
  }
}
```

Considerando o código acima, marque a opção que apresenta o pior e melhor caso para o número de multiplicações, respectivamente.

☐ n, n

☐ 3(n-3), n

☒ 3(n-4), (n-4)

☐ 3(n-3), (n-3)

☐ 3(n-4), n

Incorreta

Pergunta 5

0 / 0,5 pts

O comando condicional *if-e/se* possibilita a escolha de um grupo de ações a serem executadas quando determinadas condições de entrada são ou não satisfeitas. O trecho de código abaixo contém uma estrutura condicional.

```
if (n < a + 3 || n > b + 4 || n > c + 1){  
    l+= 5;  
} else {  
    l+= 2; k+=3; m+=7; x += 8;  
}  
  
if (n < a + 3){  
    l+= 2; k+=3; m+=7; x += 8;  
} else {  
    l+= 5;  
}
```

Considerando o código acima, marque a opção que apresenta o melhor e pior caso, respectivamente, para o número de adições.

☐ 4 e 12

☐ 5 e 12

☐ 6 e 12

☐ 5 e 9

☒ 4 e 9

O pior caso tem nove adições e acontece quando as três condições do primeiro if são falsas e, conseqüentemente, a condição única do segundo if é falsa dado que ela é igual a primeira condição do primeiro if. Dessa forma, o teste do primeiro if realiza 3 adições e sua lista de comandos, quatro. O teste do segundo if realiza uma adição e mais uma na lista do else.

O melhor tem cinco adições e isso acontece quando a primeira condição é falsa e a segunda verdadeira. Nesse caso, o teste do primeiro if realiza duas adições e sua lista de comandos, uma. No segundo if, temos uma adição do teste e mais uma da lista do else.

Pergunta 6

0,5 / 0,5 pts

Sejam $T_1(n)$, $T_2(n)$ e $T_3(n)$ as equações que descrevem a complexidade de tempo dos algoritmos Alg1, Alg2 e Alg3, respectivamente, para entradas de tamanho n . A respeito da complexidade desses algoritmos, pode-se concluir que (POSCOMP'15 - adaptada):

- ☒ Alg1, Alg2 e Alg3 são, respectivamente, $O(n)$, $O(n^2)$ e $O(n^3)$.
- ☐ Alg2 e Alg3 são $O(n^2)$.
- ☐ Alg1 e Alg2 são $O(n)$.

☐ Alg1, Alg2 e Alg3 são, respectivamente, em , e .

☐ Alg1, Alg2 e Alg3 são, respectivamente, , e

Realmente, Alg1, Alg2 e Alg3 são, respectivamente, , e .

Incorreta

Pergunta 7

0 / 0,5 pts

Um dos problemas mais importantes na Computação é pesquisar a existência de um elemento em um conjunto de dados. Duas técnicas tradicionais de pesquisa são a sequencial e a binária. A respeito dessas técnicas assinale a alternativa correta (POSCOMP'12 - adaptado).

☐ A pesquisa sequencial percorre todos os elementos para encontrar a chave.

☒ A pesquisa binária percorre, em média, a metade dos elementos do vetor.

☐ A pesquisa binária pode ser feita sobre qualquer distribuição dos elementos.

☐ A pesquisa binária percorre no pior caso elementos.

☐ A pesquisa sequencial exige que os elementos estejam completamente ordenados.

Realmente, a pesquisa binária percorre no pior caso elementos.

Pergunta 8

0,5 / 0,5 pts

A ordenação interna é um problema clássico na Computação. Considerando-o, avalie as asserções que se seguem:

I. O algoritmo Countingsort ordena um vetor com custo linear.

PORQUE

II. O limite inferior do problema de ordenação interna é para a comparação entre registros.

A respeito dessas asserções, assinale a opção correta.

- ☐ A asserção I é uma proposição verdadeira, e a asserção II é uma proposição falsa
- ☐ As asserções I e II são proposições falsas
- ☐ A asserção I é uma proposição falsa, e a asserção II é uma proposição verdadeira

- ☒ As asserções I e II são proposições verdadeiras, mas a segunda não é uma justificativa correta da primeira
- ☐ As asserções I e II são proposições verdadeiras, e a segunda é uma justificativa correta da primeira

I - CORRETA: O algoritmo Countingsort efetua em tempo linear a ordenação dos elementos de um vetor. Ele considera três vetores: entrada, contagem e saída. O primeiro passo é em é criar o vetor de contagem de tal forma que cada posição tenha o número de elementos menores ou iguais aquela posição. O segundo passo é copiar cada elemento do vetor de entrada para o de saída mapeando de tal forma que a posição do elemento no vetor de saída será mapeada a partir do vetor de contagem.

II - CORRETA: É impossível ordenar um vetor com menos do que comparações entre os elementos do vetor. O Countingsort não se aplica a tal regra porque ele triplica o espaço de memória e não funciona para qualquer tipo de elemento.

As duas afirmações são independentes.

Pergunta 9

0,5 / 0,5 pts

A primeira fase do heapsort constroi um *heap* com os elementos do vetor. Seja o vetor [20, 10, 5, 30, 50, 45, 35] onde o primeiro elemento (20) está na posição 1, assinale a opção que contém o heap construído no final da fase citada (INMETRO'10, adaptada).

☐ [20, 10, 30, 5, 15, 45, 50]

☐ [50, 20, 45, 30, 10, 5, 35]

☐ [5, 10, 20, 30, 35, 45, 50]

☒ [50, 30, 45, 10, 20, 5, 35]

☐ [50, 45, 35, 30, 20, 15, 10]

Aplicando o algoritmo do Heapsort, temos a sequência de resposta.

Incorreta

Pergunta 10

0 / 0,5 pts

Um algoritmo clássico para a ordenação de elementos de um vetor é o Ordenação por Seleção. Abaixo, temos uma implementação desse algoritmo.

```
for (i = 0; i < (n - 1); i++) {  
    indice = i;  
    for (j = (i + 1); j < n; j++){  
        if (vet[indice] > vet[j]){  
            indice = j;  
        }  
    }  
}
```

```
tmp = vet[indice];  
vet[indice] = vet[i];  
vet[i] = tmp;  
}
```

Considerando o código acima e seus conhecimentos sobre algoritmos de ordenação, avalie as asserções que se seguem:

I. O algoritmo de Seleção deve ser considerado como uma opção para ordenação quando tivermos registros grandes".

PORQUE

II. O algoritmo de Seleção possui complexidade quadrática - para o número de comparações envolvendo os elementos do vetor.

A respeito dessas asserções, assinale a opção correta.

-
- ☐ A asserção I é uma proposição verdadeira, e a asserção II é uma proposição falsa.
-
- ☐ As asserções I e II são proposições verdadeiras, e a segunda é uma justificativa correta da primeira.
-
- ☐ As asserções I e II são proposições falsas.
-
- ☒ A asserção I é uma proposição falsa, e a asserção II é uma proposição verdadeira.
-
- ☐ As asserções I e II são proposições verdadeiras, mas a segunda não é uma justificativa correta da primeira.

Realmente, as duas afirmações são verdadeiras, contudo, um não justifica a outra.

Incorreta

Pergunta 11

0 / 0,5 pts

Sobre a escolha adequada para um algoritmo de ordenação, considere as afirmativas a seguir.

- I. Quando os cenários de pior caso for a preocupação, o algoritmo ideal é o Heap Sort.
- II. Quando o vetor apresenta a maioria dos elementos ordenados, o algoritmo ideal é o Insertion Sort.
- III. Quando o interesse for um bom resultado para o médio caso, o algoritmo ideal é o Quick Sort.
- IV. Quando o interesse é o melhor caso e o pior caso de mesma complexidade, o algoritmo ideal é o Bubble Sort.

Assinale a alternativa correta

- ☐ Somente as afirmativas III e IV são corretas
- ☐ Somente as afirmativas I, II e III são corretas.
- ☐ Somente as afirmativas I e II são corretas.
- ☐ Somente as afirmativas I e IV são corretas.

☐ Somente as afirmativas II, III e IV são corretas.

(POSCOMP'13)

Incorreta

Pergunta 12

0 / 0,5 pts

Um dos principais algoritmos de ordenação é o Quicksort. Considerando esse algoritmo, avalie as afirmações a seguir (TRT-PR'13, adaptado)

- I. Considere o método de ordenação que implementa o seguinte processo: uma coleção de elementos é dividida em duas partes. Uma contendo os elementos da coleção que são menores ou iguais a um dado pivô e outra com os maiores ou iguais. Em seguida, submetemos cada metade à mesma subrotina de partição. Repetimos esse processo até que cada parte tenha tamanho 1, resultando em uma coleção ordenada. A complexidade do caso médio desse algoritmo é expressa por .
- II. Existem aplicações para listas lineares nas quais inserções, retiradas e acessos a itens ocorrem sempre em um mesmo extremo da lista. Nesses casos, a estrutura adequada para resolvê-los é a pilha ou *stack*.
- III. No método Quicksort, o pivô é responsável pelo número de partições em que o vetor é dividido. Como o pivô não pode ser um elemento que esteja repetido no vetor, o Quicksort não funciona quando há elementos repetidos.

É correto apenas o que se afirma em

☐ II e III.☐ I e II.☒ I.☐ II.☐ I e III.

I - CORRETA: A alternativa descreve corretamente o funcionamento básico do *quicksort* e apresenta também sua complexidade para o número de comparações entre elementos do vetor.

II - CORRETA: A alternativa descreve uma TAD pilha.

III - ERRADA: No quicksort, dividimos o vetor sistematicamente em duas partes. Além disso, em uma partição, os elementos iguais ao pivô podem ficar em qualquer um dos dois lados.

Pergunta 13

2 / 2 pts

Encontre a fórmula fechada do somatório matemática.

e, em seguida, prove a usando indução

↓ [Prova 1 - Questão 13.pdf \(https://pucminas.instructure.com/files/4543966/download\)](https://pucminas.instructure.com/files/4543966/download)

A ser explicada na aula posterior à prova.

Pergunta 14

2 / 2 pts

Seja nossa classe **Fila circular**, faça um método **R.E.C.U.R.S.I.V.O** que recebe três objetos do tipo Fila circular e mostra na tela a soma do primeiro elemento das três listas, a do segundo, do terceiro e, assim, sucessivamente. Lembre-se que as filas podem ter tamanhos distintos.

↓ [P01Q13.java \(https://pucminas.instructure.com/files/4544436/download\)](https://pucminas.instructure.com/files/4544436/download)

Nesta questão o aluno deve *desenfileirar* e exibir um elemento de cada fila, intercalando. Assim, caso uma das filas fique vazia primeiro, a exibição deve continuar até que a outra estrutura fique vazia. Atenção as consistências que devem ser feitas para cada estrutura, verificando se a mesma esta ou não vazia.

Pontuação do teste: **7,5** de 10